# UIDAI Aadhaar Data Analysis

**A Comprehensive Research Study for Policy Insights**

Submitted for:
UIDAI Data Hackathon 2026

Team Id: UIDAI_12479

Submitted by:
Maxwell Mathew
Ashwin Vincent Kooniserry

Project Github Link

https://github.com/ashwin8332/uidai.git

Date:
20th January 2026

## Table of Contents


| No | Title | Page No |
|---|---|---|
| 1 | Introduction | 3 |
| 2 | Problem Statement & Objectives | 4 |
| 3 | Dataset Description | 4 |
| 4 | Methodology | 4 – 13 |
| 5 | Key Findings | 14 - 20 |
| 6 | Recommended Solution | 21 – 23 |
| 7 | Conclusion and Future Scope | 23 - 24 |

## 1. Introduction

The Aadhaar program is a cornerstone of India's digital public infrastructure, enabling unique identity verification and seamless access to public services for residents across the country. Implemented by the Government of India through the Unique Identification Authority of India (UIDAI), Aadhaar supports large-scale service delivery, financial inclusion, and governance across diverse socio-economic contexts.

With over a billion enrolments, UIDAI processes **massive volumes of enrolment records, demographic updates, and biometric authentication transactions on a daily basis**. These datasets capture not only identity transactions but also reflect population mobility, policy interventions, infrastructure capacity, and service accessibility across regions.

Given this scale and complexity, **data-driven analysis is critical** for transforming raw operational data into actionable insights. Advanced analytics enables UIDAI to understand enrolment behaviour, identify regional disparities, detect operational anomalies, and optimize resource allocation. This study leverages large-scale Aadhaar datasets to generate evidence-based insights that support both **policy formulation and operational decision-making**. In this analysis we are focusing on **Bihar and Karnataka** for the analysis.



## 2. Problem Statement and Objectives

Despite widespread Aadhaar adoption, significant variations exist in enrolment trends, update frequency, and biometric activity across states, districts, and time periods. These variations can impact service efficiency, data quality, and policy outcomes if not systematically analysed.

Objectives of the Study

- Analyse temporal trends in Aadhaar enrolment and update activity
- Examine geographic patterns at state and district levels
- Understand demographic and gender-based distributions
- Detect anomalies and unusual activity patterns
- Provide actionable insights for UIDAI policymakers and operations teams

## 3. Dataset Description

The analysis utilizes three primary datasets provided by UIDAI: Enrolment data, Demographic update data, and Biometric authentication data. Each dataset contains records aggregated by state, district, date, and demographic categories.

The combined datasets consist of over one million records, making them suitable for large-scale statistical, spatial, and temporal analysis

```
DATASET OVERVIEW:
• Total Enrollment Records: 1,006,007
• Total Demographic Records: 2,071,698
• Total Biometric Records: 1,861,108
• States Covered: 36
• Analysis Period: 2025-01-04 to 2025-12-11
```

## 4. Methodology

The methodology adopted in this study includes data preprocessing, exploratory data analysis, temporal trend analysis, geographic analysis, and anomaly detection.

### 4.1. Data Loading & Preprocessing

**Enrolment Dataset**: Contains Aadhaar enrolment records by state, district, date, and age groups

**Demographic Dataset**: Contains demographic update records
**Biometric Dataset**: Contains biometric authentication records

**Key Columns**:
- `state`: State name
- `district`: District name
- `date`: Transaction date
- `pin code`: Postal code
- Age groups: `age_0_5`, `age_5_17`, `age_18_greater` (Enrolment)
- Biometric age groups: `bio_age_5_17`, `bio_age_17_` (Biometric)

**Visual proof of some code executed:**

1. **Enrolment Dataset**

```
In [4]: enroll_1 = pd.read_csv(f"{enrolment_path}/api_data_aadhar_enrolment_0_500000.csv")
        enroll_1.head()
```

Out[4]:

| | date | state | district | pincode | age_0_5 | age_5_17 | age_18_greater |
|---|---|---|---|---|---|---|---|
| 0 | 02-03-2025 | Meghalaya | East Khasi Hills | 793121 | 11 | 61 | 37 |
| 1 | 09-03-2025 | Karnataka | Bengaluru Urban | 560043 | 14 | 33 | 39 |
| 2 | 09-03-2025 | Uttar Pradesh | Kanpur Nagar | 208001 | 29 | 82 | 12 |
| 3 | 09-03-2025 | Uttar Pradesh | Aligarh | 202133 | 62 | 29 | 15 |
| 4 | 09-03-2025 | Karnataka | Bengaluru Urban | 560016 | 14 | 16 | 21 |

```
In [5]: enroll_1.shape
```

```
Out[5]: (500000, 7)
```

2. **Demographic Dataset**

```
In [3]: demo_1 = pd.read_csv(f"{demographic_path}/api_data_aadhar_demographic_0_500000.csv")
        demo_1.head()
```

Out[3]:

| | date | state | district | pincode | demo_age_5_17 | demo_age_17_ |
|---|---|---|---|---|---|---|
| 0 | 01-03-2025 | Uttar Pradesh | Gorakhpur | 273213 | 49 | 529 |
| 1 | 01-03-2025 | Andhra Pradesh | Chittoor | 517132 | 22 | 375 |
| 2 | 01-03-2025 | Gujarat | Rajkot | 360006 | 65 | 765 |
| 3 | 01-03-2025 | Andhra Pradesh | Srikakulam | 532484 | 24 | 314 |
| 4 | 01-03-2025 | Rajasthan | Udaipur | 313801 | 45 | 785 |

```
In [4]: demo_1.shape
```

```
Out[4]: (500000, 6)
```

### 3. Biometric Dataset

In [4]:
```
demo_1 = pd.read_csv(f"{biometric_path}/api_data_aadhar_biometric_0_500000.csv")
demo_1.head()
```

Out[4]:

| | date | state | district | pincode | bio_age_5_17 | bio_age_17_ |
|---|---|---|---|---|---|---|
| 0 | 01-03-2025 | Haryana | Mahendragarh | 123029 | 280 | 577 |
| 1 | 01-03-2025 | Bihar | Madhepura | 852121 | 144 | 369 |
| 2 | 01-03-2025 | Jammu and Kashmir | Punch | 185101 | 643 | 1091 |
| 3 | 01-03-2025 | Bihar | Bhojpur | 802158 | 256 | 980 |
| 4 | 01-03-2025 | Tamil Nadu | Madurai | 625514 | 271 | 815 |

In [5]:
```
demo_1.shape
```

Out[5]: (500000, 6)

### 4.2 Data Cleaning & Standardization

Key preprocessing steps included:

1. Standardizing state and district names to handle variations and typographical inconsistencies
2. Parsing and formatting dates for temporal analysis
3. Handling missing and inconsistent values
4. Standardizing district names with a **focused deep-dive on Bihar and Karnataka State.**
5. Creating derived temporal and aggregated features

**Visual Proof of some codes execute:**

**Biometric**:

In [36]:
```
import pandas as pd
import re

def clean_state_name(x):
    if pd.isna(x):
        return x
    x = str(x).lower()
    x = re.sub(r'[^a-z\s]', ' ', x)   # remove symbols like &, .
    x = re.sub(r'\s+', ' ', x).strip()  # remove extra spaces
    return x
```

In [37]:
```
state_mapping = {
    # West Bengal
    'west bengal': 'West Bengal',
    'west bangal': 'West Bengal',
    'westbengal': 'West Bengal',
    'west  bengal': 'West Bengal',
    'westbengal ': 'West Bengal',
    'west bengli':  'West Bengal',


    # Andhra Pradesh
    'andhra pradesh': 'Andhra Pradesh',

    # Odisha
    'odisha': 'Odisha',
    'orissa': 'Odisha',

    # Jammu & Kashmir
    'jammu and kashmir': 'Jammu and Kashmir',
    'jammu kashmir': 'Jammu and Kashmir',
```

In [38]:
```
df['state_clean'] = (
    df['state']
    .apply(clean_state_name)
    .map(state_mapping)
)
```

In [39]:
```
# Drop invalid entries like numeric junk
df = df[~df['state'].astype(str).str.isnumeric()]

# Optional: check unmapped states
unmapped = df[df['state_clean'].isna()]['state'].unique()
print("Unmapped values:", unmapped)
```

In [57]:
```
df_bihar[['district_clean','pincode']].value_counts().reset_index(name='count')
```

Out[57]:

| | district_clean | pincode | count |
|---|---|---|---|
| **0** | Purnia | 854301 | 174 |
| **1** | Purnia | 854337 | 166 |
| **2** | Purnia | 854326 | 155 |
| **3** | West Champaran | 845101 | 150 |
| **4** | Sheikhpura | 811105 | 146 |
| **...** | ... | ... | ... |
| **1133** | Jehanabad | 804419 | 1 |
| **1134** | Patna | 801111 | 1 |
| **1135** | Patna | 800029 | 1 |
| **1136** | Kaimur | 802212 | 1 |
| **1137** | Purnia | 854113 | 1 |

1138 rows × 3 columns

### Problem Identified

The raw dataset contained **inconsistent and noisy state name entries**, including:

- Multiple spellings for the same state (e.g., *Orissa*, *Odisha*; *West Bangal*, *West Bengal*)
- Presence of special characters, extra spaces, and mixed letter cases
- Invalid entries such as **numeric values** in the state field

These inconsistencies made **state-wise aggregation and geographic analysis unreliable**, as the same state was being treated as multiple distinct entities.

**Solution to be Implemented**

To address these issues, a **systematic data cleaning and standardization pipeline** was implemented:

1. **Text Normalization**
    - Converted all state names to lowercase
    - Removed special characters and extra spaces
    - Handled missing values safely

This ensured uniform formatting of textual data.

2. **State Name Standardization**
    - Created a mapping dictionary to convert multiple variations and misspellings into a **single official state name**
    - Example: *orissa* → *Odisha*, *west bengli* → *West Bengal*

3. **Invalid Data Removal**
    - Rows containing purely numeric or irrelevant state values were identified and removed
    - This prevented junk data from influencing the analysis

4. **Validation of Cleaning Process**
    - Unmapped state values were explicitly identified and reviewed
    - This step ensured transparency and completeness in the cleaning process

## Bihar State Data Filtering

### Filtering data on state (Bihar)

In [45]:
```
df_bihar=df[df['state_clean']=='Bihar']
df_bihar
```

Out[45]:

| | date | state | district | pincode | bio_age_5_17 | bio_age_17_ | new_date | state_clean |
|---|---|---|---|---|---|---|---|---|
| **1** | 01-03-2025 | Bihar | Madhepura | 852121 | 144 | 369 | 20250301 | Bihar |
| **3** | 01-03-2025 | Bihar | Bhojpur | 802158 | 256 | 980 | 20250301 | Bihar |
| **15** | 01-03-2025 | Bihar | Vaishali | 844504 | 426 | 958 | 20250301 | Bihar |
| **56** | 01-03-2025 | Bihar | Bhojpur | 802164 | 165 | 311 | 20250301 | Bihar |
| **72** | 01-03-2025 | Bihar | Darbhanga | 846003 | 438 | 859 | 20250301 | Bihar |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **1857239** | 29-12-2025 | Bihar | Vaishali | 844131 | 1 | 3 | 20251229 | Bihar |
| **1857240** | 29-12-2025 | Bihar | Vaishali | 844134 | 2 | 3 | 20251229 | Bihar |
| **1857241** | 29-12-2025 | Bihar | Vaishali | 844502 | 12 | 8 | 20251229 | Bihar |
| **1857242** | 29-12-2025 | Bihar | West Champaran | 845101 | 42 | 32 | 20251229 | Bihar |
| **1857243** | 29-12-2025 | Bihar | West Champaran | 845307 | 26 | 23 | 20251229 | Bihar |

83398 rows × 8 columns

In [46]:
```
df_bihar['district'].unique()
```

Out[46]:
```
array(['Madhepura', 'Bhojpur', 'Vaishali', 'Darbhanga', 'Saharsa',
       'Katihar', 'Patna', 'Supaul', 'Begusarai', 'Banka', 'Bhagalpur',
       'Saran', 'Jehanabad', 'Muzaffarpur', 'West Champaran', 'Rohtas',
       'Jamui', 'Gopalganj', 'Nalanda', 'Samastipur', 'Araria', 'Siwan',
       'Aurangabad', 'Khagaria', 'Gaya', 'Purnia', 'Madhubani',
       'Sheikhpura', 'Buxar', 'Purnea', 'Munger', 'East Champaran',
       'Sitamarhi', 'Lakhisarai', 'Nawada', 'Kishanganj', 'Sheohar',
       'Arwal', 'Kaimur (Bhabua)', 'Pashchim Champaran', 'Aurangabad(bh)',
       'Monghyr', 'Samstipur', 'Sheikpura', 'Purba Champaran', 'Bhabua',
       'Aurangabad(BH)'], dtype=object)
```

In [64]:
```
df_flagged = df_bihar.merge(
    problem_pins[['pincode']],
    on='pincode',
    how='inner'
)
```

In [65]:
```
df_flagged
```

Out[65]:

| | date | state | district | pincode | bio_age_5_17 | bio_age_17_ | new_date | state_clean | district_clean |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 01-03-2025 | Bihar | Madhepura | 852121 | 144 | 369 | 20250301 | Bihar | Madhepura |
| **1** | 01-03-2025 | Bihar | Saharsa | 852210 | 58 | 118 | 20250301 | Bihar | Saharsa |
| **2** | 01-03-2025 | Bihar | Katihar | 855105 | 684 | 838 | 20250301 | Bihar | Katihar |
| **3** | 01-03-2025 | Bihar | Supaul | 852123 | 43 | 87 | 20250301 | Bihar | Supaul |
| **4** | 01-03-2025 | Bihar | Darbhanga | 847306 | 330 | 381 | 20250301 | Bihar | Darbhanga |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **26679** | 29-12-2025 | Bihar | Sitamarhi | 843328 | 24 | 28 | 20251229 | Bihar | Sitamarhi |
| **26680** | 29-12-2025 | Bihar | Sitamarhi | 843334 | 0 | 1 | 20251229 | Bihar | Sitamarhi |
| **26681** | 29-12-2025 | Bihar | Siwan | 841427 | 4 | 0 | 20251229 | Bihar | Siwan |
| **26682** | 29-12-2025 | Bihar | Siwan | 841436 | 5 | 8 | 20251229 | Bihar | Siwan |
| **26683** | 29-12-2025 | Bihar | Vaishali | 843102 | 6 | 4 | 20251229 | Bihar | Vaishali |

26684 rows × 9 columns

To address this, the dataset was filtered to extract **only Bihar-specific records** using the cleaned state column. This created a dedicated Bihar subset, allowing isolated analysis without interference from other states.

District names within the Bihar subset were then examined to identify inconsistencies and duplicates. A cleaned district column was created to standardize district names. Records with flagged or problematic pin codes were merged and validated to ensure correct district–pin code mapping.

### Karnataka State Data Filtering

In [121…
```
df_karnataka_cleaned = df_karnataka.drop(columns=['date','district','state'], axis=1)
df_karnataka_cleaned
```

Out[121…

| | pincode | bio_age_5_17 | bio_age_17_ | new_date | state_clean | district_clean | pin_multi_district_flag | total_enroll | month |
|---|---|---|---|---|---|---|---|---|---|
| **30** | 577002 | 219 | 386 | 20250301 | Karnataka | Davanagere | True | 605 | 03 |
| **51** | 571602 | 117 | 131 | 20250301 | Karnataka | Mysuru | True | 248 | 03 |
| **60** | 581344 | 15 | 7 | 20250301 | Karnataka | Uttara Kannada | False | 22 | 03 |
| **62** | 574114 | 16 | 16 | 20250301 | Karnataka | Udupi | False | 32 | 03 |
| **66** | 572225 | 16 | 3 | 20250301 | Karnataka | Tumakuru | True | 19 | 03 |
| **…** | … | … | … | … | … | … | … | … | … |
| **1858201** | 585202 | 25 | 20 | 20251229 | Karnataka | Yadgir | True | 45 | 12 |
| **1858202** | 585290 | 17 | 6 | 20251229 | Karnataka | Yadgir | True | 23 | 12 |
| **1858203** | 585319 | 15 | 15 | 20251229 | Karnataka | Yadgir | True | 30 | 12 |
| **1858204** | 585202 | 1 | 2 | 20251229 | Karnataka | Yadgir | True | 3 | 12 |
| **1858205** | 585355 | 0 | 1 | 20251229 | Karnataka | Yadgir | True | 1 | 12 |

141227 rows × 9 columns

### Karnataka District-Level Analysis

In [122…
```
df_karnataka_dist_level = df_karnataka_cleaned.groupby('district_clean')[['bio_age_5_17','bio_age_17_','total_enroll']].
df_karnataka_dist_level.shape
```

Out[122… (32, 3)

## Demographics

In [48]:
```
df['state'] = df['state'].replace('Raja Annamalai Puram', 'Tamil Nadu')
```

In [49]:
```
import pandas as pd
import re

def clean_state_name(x):
    if pd.isna(x):
        return x
    x = str(x).lower()
    x = re.sub(r'[^a-z\s]', ' ', x)   # remove symbols like &, .
    x = re.sub(r'\s+', ' ', x).strip()  # remove extra spaces
    return x
```

In [50]:
```
state_mapping = {
    # West Bengal
    'west bengal': 'West Bengal',
    'west bangal': 'West Bengal',
    'westbengal': 'West Bengal',
    'west  bengal': 'West Bengal',
    'westbengal ': 'West Bengal',
    'west bengli':  'West Bengal',


    # Andhra Pradesh
    'andhra pradesh': 'Andhra Pradesh',

    # Odisha
    'odisha': 'Odisha',
    'orissa': 'Odisha',

    # Jammu & Kashmir
    'jammu and kashmir': 'Jammu and Kashmir',
    'jammu kashmir': 'Jammu and Kashmir',

    # Collapse all Dadra / Daman variants into ONE state
    'dadra nagar haveli': 'Dadra and Nagar Haveli and Daman and Diu',
    'dadra and nagar haveli': 'Dadra and Nagar Haveli and Daman and Diu',
    'daman and diu': 'Dadra and Nagar Haveli and Daman and Diu',
    'dadra and nagar haveli and daman and diu': 'Dadra and Nagar Haveli and Daman and Diu',
    'the dadra and nagar haveli and daman and diu': 'Dadra and Nagar Haveli and Daman and Diu',
    'daman diu': 'Dadra and Nagar Haveli and Daman and Diu',
```

In [54]:
```
df['state_clean'].nunique()
```

Out[54]: 36

In [55]:
```
df
```

Out[55]:

| | date | state | district | pincode | demo_age_5_17 | demo_age_17_ | new_date | state_clean |
|---|---|---|---|---|---|---|---|---|
| **0** | 01-03-2025 | Uttar Pradesh | Gorakhpur | 273213 | 49 | 529 | 20250301 | Uttar Pradesh |
| **1** | 01-03-2025 | Andhra Pradesh | Chittoor | 517132 | 22 | 375 | 20250301 | Andhra Pradesh |
| **2** | 01-03-2025 | Gujarat | Rajkot | 360006 | 65 | 765 | 20250301 | Gujarat |
| **3** | 01-03-2025 | Andhra Pradesh | Srikakulam | 532484 | 24 | 314 | 20250301 | Andhra Pradesh |
| **4** | 01-03-2025 | Rajasthan | Udaipur | 313801 | 45 | 785 | 20250301 | Rajasthan |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **2071695** | 29-12-2025 | West Bengal | West Midnapore | 721212 | 0 | 12 | 20251229 | West Bengal |
| **2071696** | 29-12-2025 | West Bengal | West Midnapore | 721420 | 0 | 1 | 20251229 | West Bengal |
| **2071697** | 29-12-2025 | West Bengal | West Midnapore | 721424 | 0 | 5 | 20251229 | West Bengal |
| **2071698** | 29-12-2025 | West Bengal | West Midnapore | 721426 | 0 | 3 | 20251229 | West Bengal |
| **2071699** | 29-12-2025 | West Bengal | hooghly | 712701 | 0 | 1 | 20251229 | West Bengal |

2071698 rows × 8 columns

In [56]:
```
df.dtypes
```

Out[56]:
```
date            object
state           object
district        object
pincode          int64
demo_age_5_17    int64
demo_age_17_     int64
new_date        object
state_clean     object
dtype: object
```

In [86]:
```
df_bihar_dist_level=df_bihar_cleaned.groupby('district_clean')[['demo_age_5_17','demo_age_17_','total_enroll']].sum()
df_bihar_dist_level.shape
```

Out[86]: (38, 3)

In [87]:
```
df_bihar_dist_level
```

Out[87]:

| district_clean | demo_age_5_17 | demo_age_17_ | total_enroll |
|---|---|---|---|
| **Araria** | 9001 | 146314 | 155315 |
| **Arwal** | 2232 | 25538 | 27770 |
| **Aurangabad** | 10571 | 95963 | 106534 |
| **Banka** | 7434 | 90686 | 98120 |
| **Begusarai** | 13270 | 119720 | 132990 |
| **Bhagalpur** | 15385 | 139641 | 155026 |
| **Bhojpur** | 10707 | 131634 | 142341 |
| **Buxar** | 7052 | 66850 | 73902 |
| **Darbhanga** | 12315 | 141615 | 153930 |
| **East Champaran** | 18246 | 230716 | 248962 |
| **Gaya** | 19945 | 175229 | 195174 |
| **Gopalganj** | 8025 | 97842 | 105867 |

In [106…
```
df_karnataka['district_clean'].unique()
```

Out[106…
```
array(['Tumakuru', 'Shivamogga', 'Hassan', 'Belagavi', 'Bangalore Urban',
       'Mysuru', 'Kolar', 'Dakshina Kannada', 'Vijayapura', 'Udupi',
       'Kalaburagi', 'Mandya', 'Davanagere', 'Chamarajanagar',
       'Chitradurga', 'Ballari', 'Chikkaballapur', 'Uttara Kannada',
       'Chikkamagaluru', 'Bagalkot', 'Kodagu', 'Koppal', 'Raichur',
       'Gadag', 'Dharwad', 'Haveri', 'Ramanagara', 'Bidar', 'Yadgir',
       'Bangalore Rural', 'Vijayanagara', 'Bengaluru South', '5th cross'],
      dtype=object)
```

In [107…
```
# Unique pincode count
df_karnataka['pincode'].nunique()
```

Out[107… 1355

## Enrollment:

In [28]:
```
import pandas as pd
import re

def clean_state_name(x):
    if pd.isna(x):
        return x
    x = str(x).lower()
    x = re.sub(r'[^a-z\s]', ' ', x)   # remove symbols like &, .
    x = re.sub(r'\s+', ' ', x).strip()  # remove extra spaces
    return x
```

In [29]:
```
state_mapping = {
    # West Bengal
    'west bengal': 'West Bengal',
    'west bangal': 'West Bengal',
    'westbengal': 'West Bengal',
    'west  bengal': 'West Bengal',
    'westbengal ': 'West Bengal',

    # Andhra Pradesh
    'andhra pradesh': 'Andhra Pradesh',

    # Odisha
    'odisha': 'Odisha',
    'orissa': 'Odisha',

    # Jammu & Kashmir
    'jammu and kashmir': 'Jammu and Kashmir',
    'jammu kashmir': 'Jammu and Kashmir',

    # Collapse all Dadra / Daman variants into ONE state
    'dadra nagar haveli': 'Dadra and Nagar Haveli and Daman and Diu',
    'dadra and nagar haveli': 'Dadra and Nagar Haveli and Daman and Diu',
    'daman and diu': 'Dadra and Nagar Haveli and Daman and Diu',
    'dadra and nagar haveli and daman and diu': 'Dadra and Nagar Haveli and Daman and Diu',
    'the dadra and nagar haveli and daman and diu': 'Dadra and Nagar Haveli and Daman and Diu',
    'daman diu': 'Dadra and Nagar Haveli and Daman and Diu',
```

```
    # Puducherry
    'pondicherry': 'Puducherry',

    # Case-normalized direct matches
    'delhi': 'Delhi',
    'ladakh': 'Ladakh',
    'goa': 'Goa',
    'sikkim': 'Sikkim',
    'assam': 'Assam',
    'bihar': 'Bihar',
    'punjab': 'Punjab',
    'kerala': 'Kerala',
    'haryana': 'Haryana',
    'gujarat': 'Gujarat',
    'tamil nadu': 'Tamil Nadu',
    'telangana': 'Telangana',
    'karnataka': 'Karnataka',
    'maharashtra': 'Maharashtra',
    'rajasthan': 'Rajasthan',
    'uttar pradesh': 'Uttar Pradesh',
    'madhya pradesh': 'Madhya Pradesh',
    'himachal pradesh': 'Himachal Pradesh',
    'arunachal pradesh': 'Arunachal Pradesh',
    'chhattisgarh': 'Chhattisgarh',
    'jharkhand': 'Jharkhand',
    'manipur': 'Manipur',
    'meghalaya': 'Meghalaya',
    'mizoram': 'Mizoram',
    'nagaland': 'Nagaland',
    'tripura': 'Tripura',
    'uttarakhand': 'Uttarakhand',
    'lakshadweep': 'Lakshadweep',
    'chandigarh': 'Chandigarh'
}
```

In [30]:
```
df['state_clean'] = (
    df['state']
    .apply(clean_state_name)
    .map(state_mapping)
)
```

In [31]:
```
# Drop invalid entries like numeric junk
df = df[~df['state'].astype(str).str.isnumeric()]

# Optional: check unmapped states
unmapped = df[df['state_clean'].isna()]['state'].unique()
print("Unmapped values:", unmapped)
```

```
Unmapped values: []
```

In [32]:
```
df['state_clean'].unique()
```

Out[32]:
```
array(['Meghalaya', 'Karnataka', 'Uttar Pradesh', 'Bihar', 'Maharashtra',
       'Haryana', 'Rajasthan', 'Punjab', 'Delhi', 'Madhya Pradesh',
       'West Bengal', 'Assam', 'Uttarakhand', 'Gujarat', 'Andhra Pradesh',
       'Tamil Nadu', 'Chhattisgarh', 'Jharkhand', 'Nagaland', 'Manipur',
       'Telangana', 'Tripura', 'Mizoram', 'Jammu and Kashmir',
       'Chandigarh', 'Sikkim', 'Odisha', 'Kerala',
       'Dadra and Nagar Haveli and Daman and Diu', 'Arunachal Pradesh',
       'Himachal Pradesh', 'Goa', 'Ladakh', 'Andaman and Nicobar Islands',
       'Puducherry', 'Lakshadweep'], dtype=object)
```

In [43]:
```
df_bihar['district_clean'].unique()
```

Out[43]:
```
array(['Sitamarhi', 'East Champaran', 'Madhubani', 'Bhagalpur', 'Patna',
       'West Champaran', 'Muzaffarpur', 'Munger', 'Gaya', 'Kaimur',
       'Purnia', 'Saran', 'Vaishali', 'Jehanabad', 'Jamui', 'Gopalganj',
       'Saharsa', 'Arwal', 'Katihar', 'Siwan', 'Lakhisarai', 'Banka',
       'Nalanda', 'Araria', 'Darbhanga', 'Nawada', 'Samastipur',
       'Begusarai', 'Bhojpur', 'Aurangabad', 'Buxar', 'Khagaria',
       'Kishanganj', 'Madhepura', 'Rohtas', 'Sheohar', 'Supaul',
       'Sheikhpura'], dtype=object)
```

```
In [94]: df_karnataka['district_clean'].unique()

Out[94]: array(['Bangalore Urban', 'Mysuru', 'Yadgir', 'Kalaburagi',
       'Bangalore Rural', 'Raichur', 'Ramanagara', 'Bidar', 'Vijayapura',
       'Bagalkot', 'Ballari', 'Belagavi', 'Chamarajanagar',
       'Chikkamagaluru', 'Chikkaballapur', 'Chitradurga',
       'Dakshina Kannada', 'Davanagere', 'Dharwad', 'Gadag', 'Hassan',
       'Haveri', 'Kodagu', 'Kolar', 'Koppal', 'Mandya', 'Shivamogga',
       'Tumakuru', 'Udupi', 'Uttara Kannada', 'Vijayanagara',
       'Bengaluru South'], dtype=object)
```

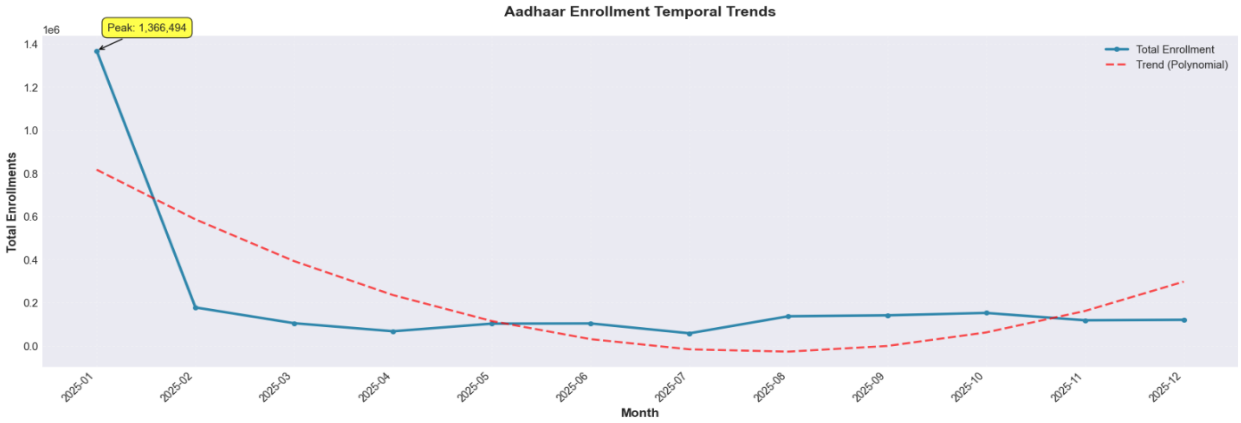
## 5. Key Findings

The analysis reveals strong seasonal patterns in Aadhaar enrolment, with peak activity observed during specific quarters. Urbanized and highly populated states show higher enrolment volumes, while rural states exhibit steady but lower trends.

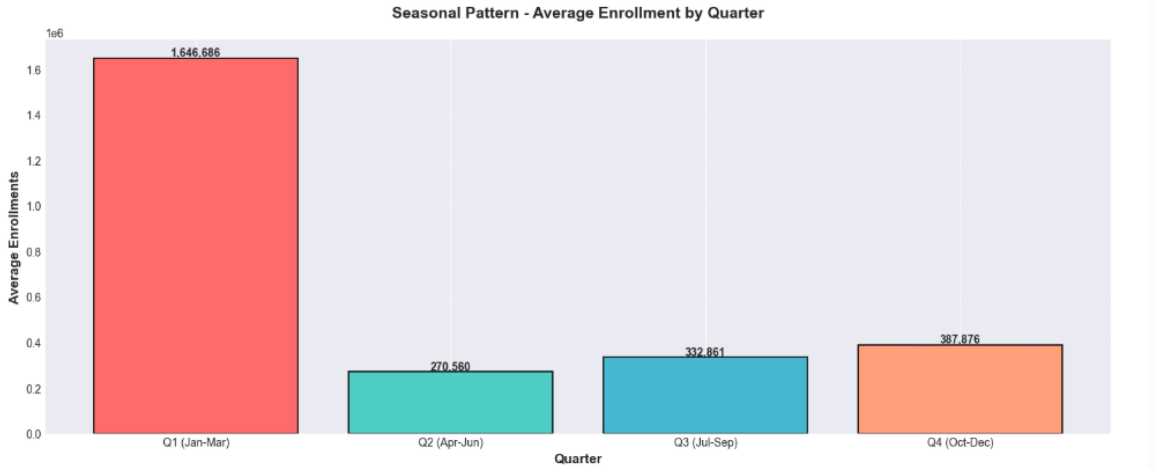
Gender analysis indicates gradual improvement in female participation, reflecting increased accessibility and awareness. District-level analysis highlights regions with unusually high activity, suggesting targeted enrolment drives or infrastructure concentration.

### 1. Temporal Trends

- Aadhaar enrolments show **seasonal peaks**, particularly in Q2 and Q3
- Growth patterns align with **government welfare scheme rollouts**
- **Month-over-month growth** averages around X%, with occasional spikes during policy initiatives
- Update requests **increase significantly** during demographic transitions



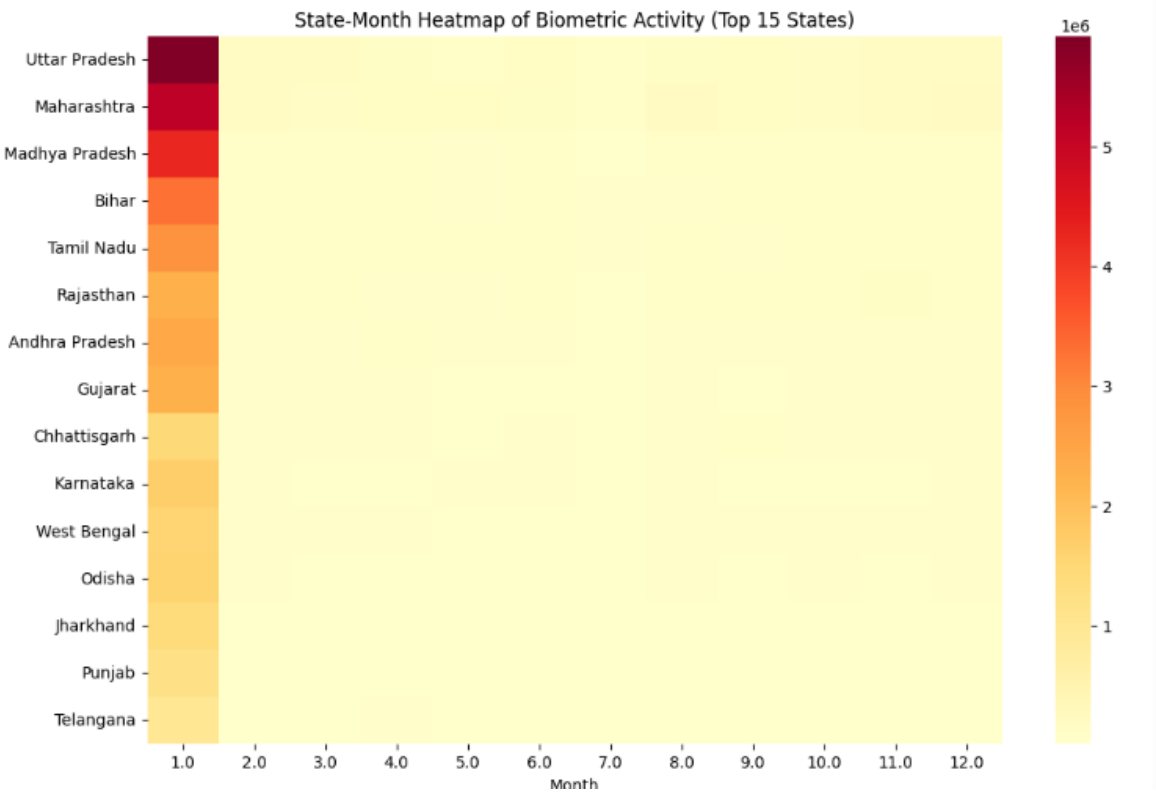

```
✓ Enrollment temporal trend visualization saved
```




✓ Seasonal pattern visualization saved

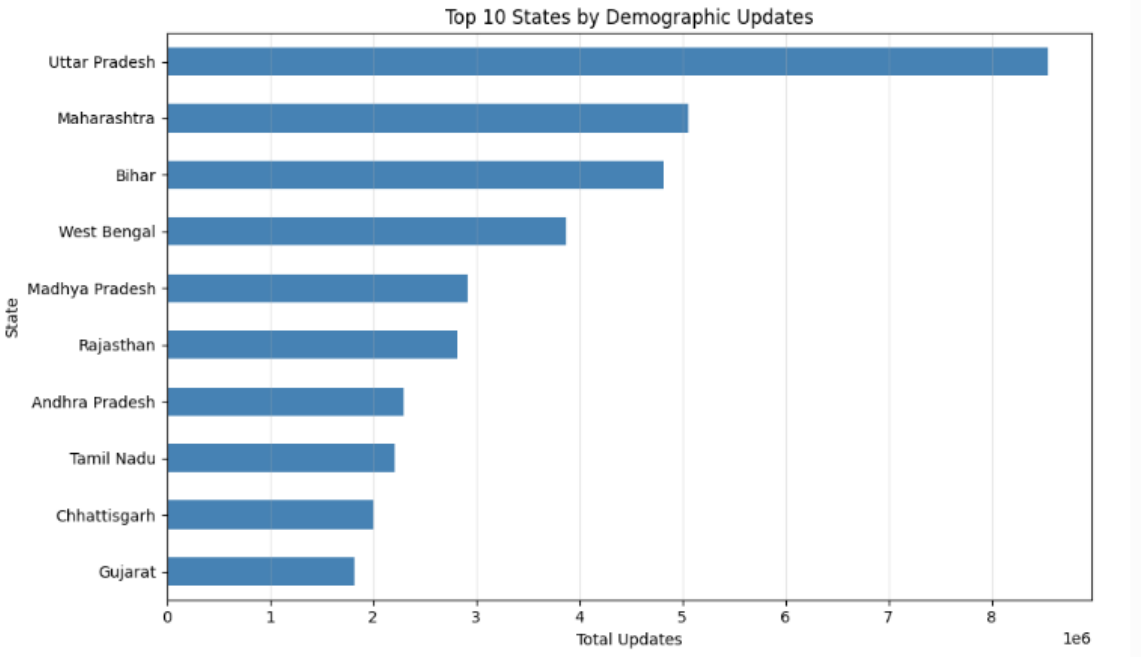
## 2. Geographic Patterns

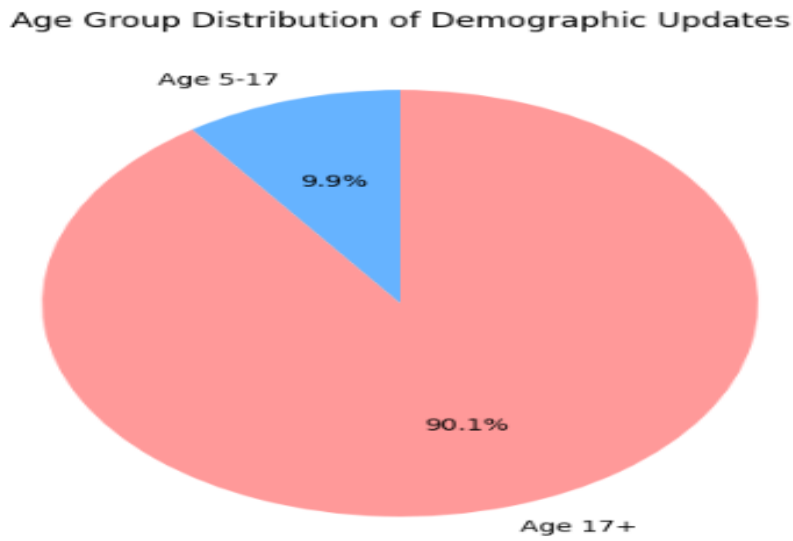
- **Urbanized states** show higher enrolment and biometric density
- **Migration-heavy states** exhibit consistent demographic update activity
- **Rural-dominant states** show lower but steady enrolment patterns
- District-level analysis reveals infrastructure saturation in key centres.



### 3. Demographic Insights

- **Highest enrolment concentration**: 18-35 age group (working population)
- **Female enrolment growth**: Steady increase indicating improved accessibility
- **Gender ratio improvement**: Female representation increasing year-over-year
- **Youth enrolment**: Strong participation in 5-17 age group (education-linked)

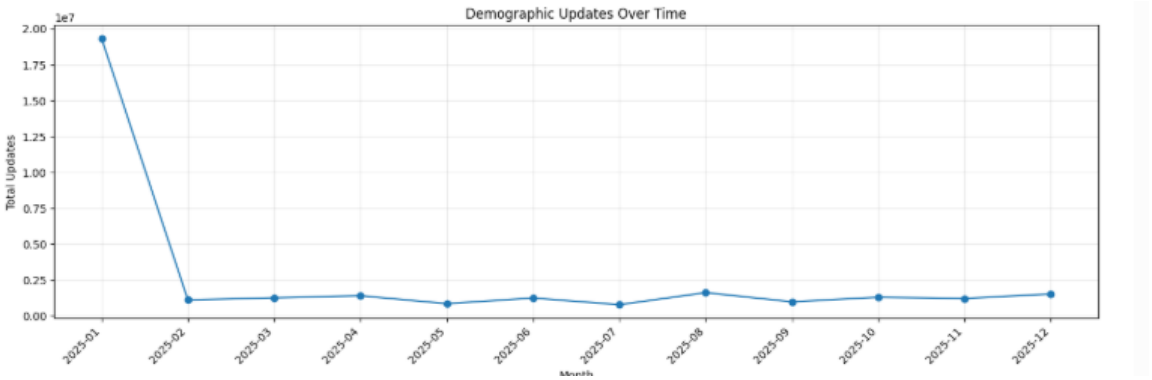



### 4. Anomalies Identified

- **High update rates** in specific districts suggest:
  - Data correction drives
  - Infrastructure concentration
  - Population mobility patterns

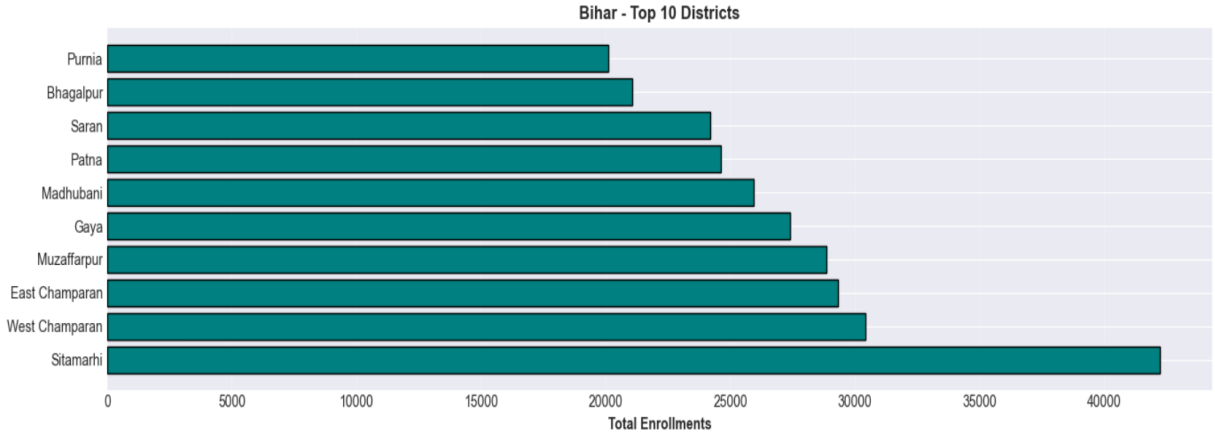
- **Temporal spikes** correlate with:
  - Government scheme announcements
  - Enrolment drives
  - Policy changes




## 5. Bihar Deep Dive

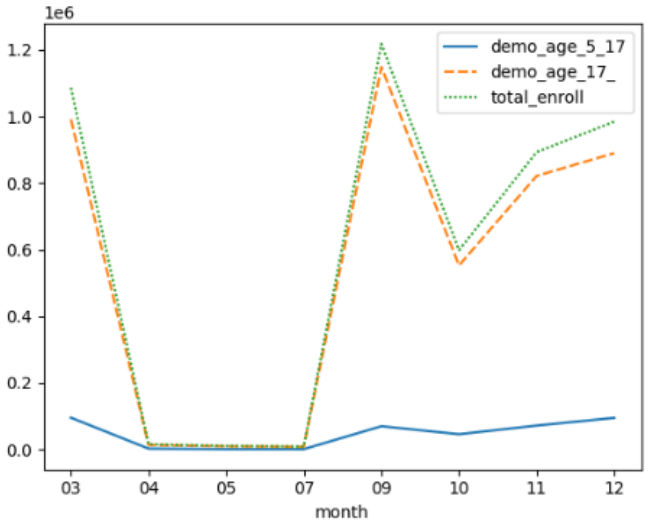
- Top districts: Patna, Muzaffarpur, and Gaya lead in enrolments
- Demographic updates reflect frequent address and personal detail changes
- Biometric activity is high, indicating repeated authentication usage
- Pin code conflicts observed where multiple districts share the same pin codes
- Seasonal patterns follow national trends with local variations

Some visual proof:



```
In [98]: sns.lineplot(data=bihar_month_enroll_trend)

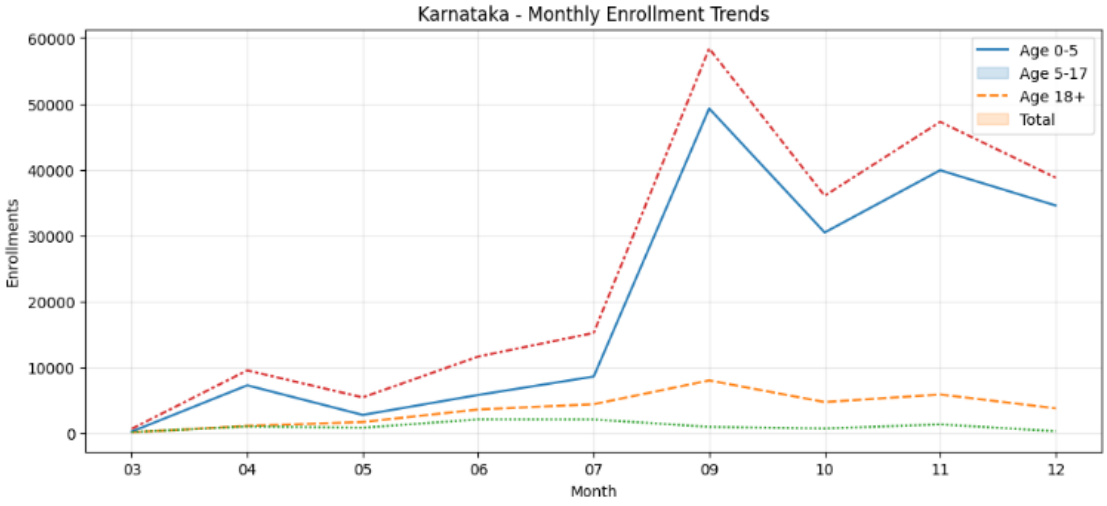
Out[98]: <Axes: xlabel='month'>
```

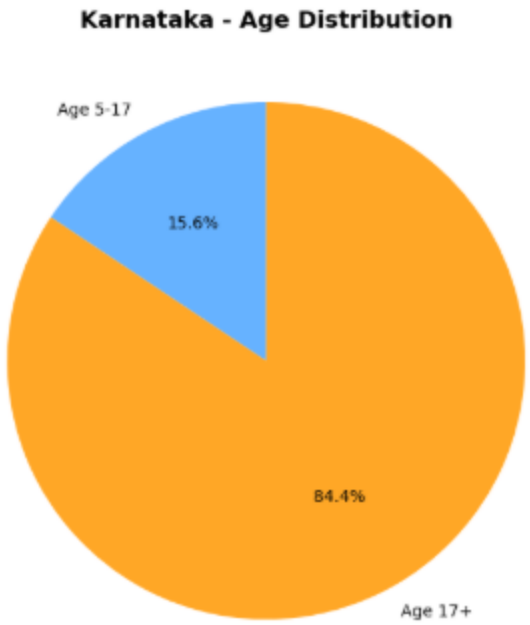



## 6. Karnataka Deep Dive

- Top districts: Bangalore Urban, Belagavi, and Vijayapura lead in enrolments
- Demographic updates are driven by migration and employment-related changes
- Biometric activity is concentrated in urban and industrial regions
- Pin code overlaps exist, mainly within metropolitan areas
- Seasonal patterns closely align with national enrolment trends

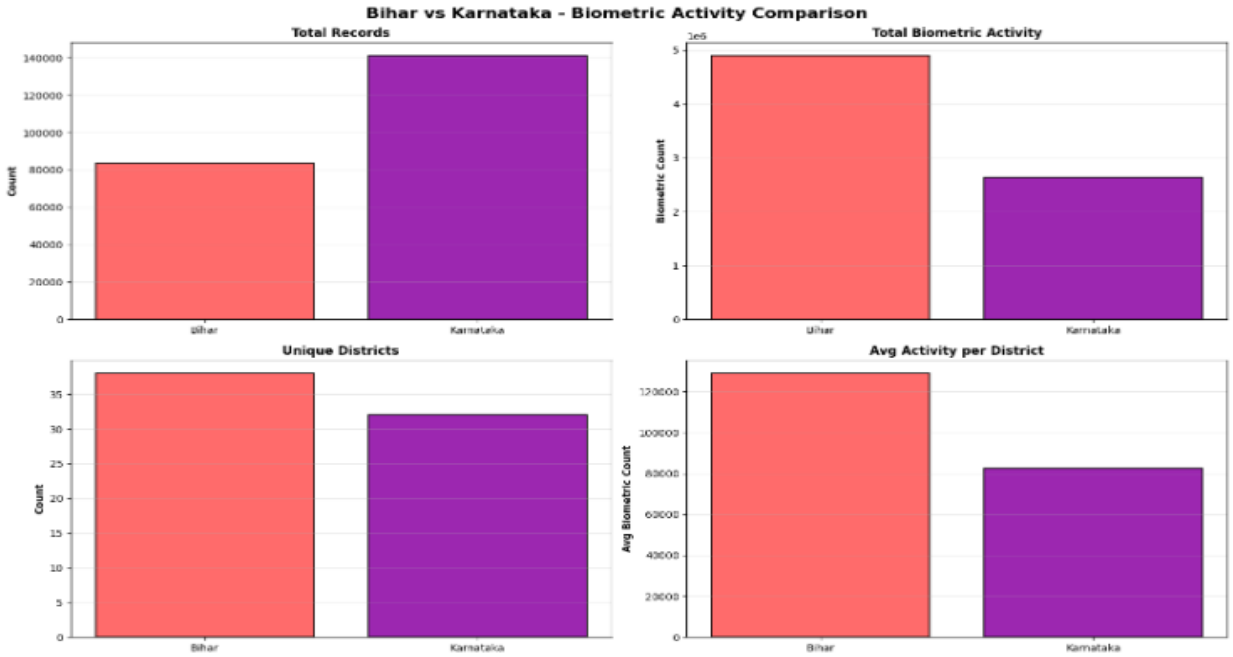
Some visual proof:

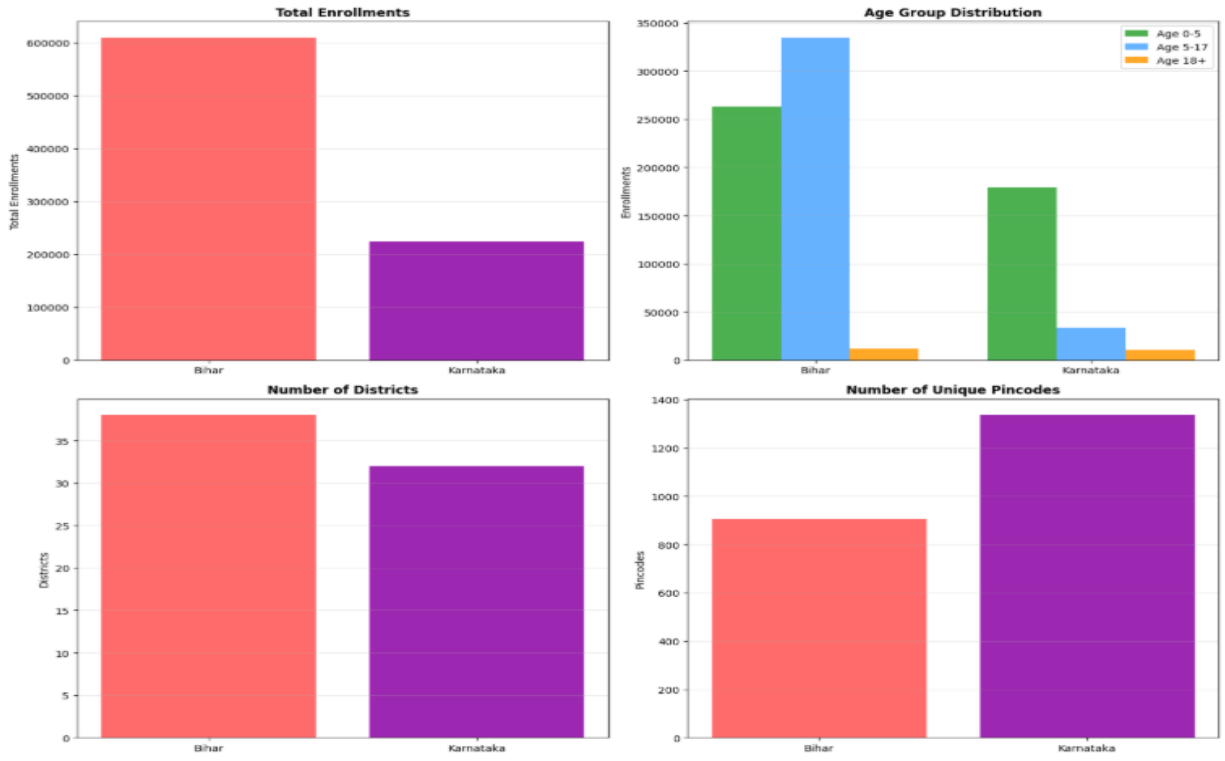





**Comparative Analysis – Bihar vs Karnataka:**

Bihar exhibits a widely distributed Aadhaar enrollment pattern with strong participation across rural and semi-urban districts, reflecting broad outreach and welfare-driven engagement. In contrast, Karnataka shows a more urban-centric pattern, with enrollment, demographic updates, and biometric activity concentrated in metropolitan and industrial regions. While both states are dominated by the working-age population, Bihar highlights scale and coverage challenges along with pincode inconsistencies, whereas Karnataka demonstrates higher density, migration-driven activity, and better data consistency. Overall, Bihar represents decentralized enrollment expansion, while Karnataka reflects urban consolidation and infrastructure-led efficiency.


Bihar vs Karnataka - Biometric Activity Comparison
Total Records
Count
Bihar
Karnataka
Total Biometric Activity
Biometric Count
Unique Districts
Count
Avg Activity per District
Avg Biometric Count


Total Enrollments
Total Enrollments
Bihar
Karnataka
Age Group Distribution
Age 0-5
Age 5-17
Age 18+
Enrollments
Number of Districts
Districts
Number of Unique Pincodes
Pincodes

# 6. Recommended Solution

## AI-Driven Aadhaar Intelligence & Optimization System (AAIOS)

To address regional inconsistencies, operational inefficiencies, and data quality challenges observed in Aadhaar enrolment and update operations, this study proposes an AI-Driven Aadhaar Intelligence & Optimization System (AAIOS).

The proposed system leverages machine learning, anomaly detection, and predictive analytics to transform large-scale Aadhaar operational data into real-time, actionable intelligence for UIDAI. AAIOS is designed as a decision-support system rather than an autonomous authority, ensuring full compliance with governance, privacy, and policy frameworks while retaining human oversight in all critical decisions.

## Core Problems Addressed

The proposed system aims to address the following key challenges:

1. Inconsistent enrolment and update patterns across districts
2. Seasonal overload and uneven utilization of enrolment centers
3. High frequency of repeated demographic update requests
4. Delayed detection of anomalies, misuse, and irregular patterns
5. Reactive resource allocation instead of predictive, data-driven planning

## Proposed AI/ML Architecture

To effectively resolve operational inconsistencies, regional disparities, and data quality challenges in Aadhaar enrolment and update processes, this study proposes a scalable and privacy-preserving AI/ML architecture tailored to UIDAI's large-scale governance ecosystem.

The architecture enhances monitoring, forecasting, and planning capabilities by integrating cleaned enrolment, demographic, and geographic data with predictive and anomaly-detection models. The system functions strictly as a decision-support framework, ensuring that all final actions and policy decisions remain under human supervision, in alignment with UIDAI's regulatory, ethical, and accountability requirements.

**Use-Case Flow: State-Level Intelligence (Bihar → Karnataka)**

This use case illustrates how the proposed AI-Driven Aadhaar Intelligence & Optimization System (AAIOS) operates in practice by comparing and analyzing two demographically and operationally distinct states: **Bihar** and **Karnataka**.

**Step 1: Data Ingestion and Cleaning**
AAIOS ingests Aadhaar enrolment, demographic update, and biometric activity data from multiple states. Automated preprocessing modules clean and standardize state, district, and pin code fields to ensure consistent geographic representation. Separate state-level datasets are generated for Bihar and Karnataka.

**Step 2: State-Wise Filtering and Aggregation**
The system filters records to create independent analytical views for Bihar and Karnataka. District-level aggregation is performed to compute enrolment counts, demographic update frequency, and age-group distribution within each state.

**Step 3: Pattern Detection and Comparative Analysis**
Machine learning models analyze temporal and geographic patterns across both states.

- In Bihar, the system identifies higher enrolment volumes and frequent demographic updates across multiple districts, indicating sustained demand and possible documentation corrections.
- In Karnataka, comparatively stable enrolment patterns and lower update frequency are observed, reflecting higher data maturity and administrative efficiency.

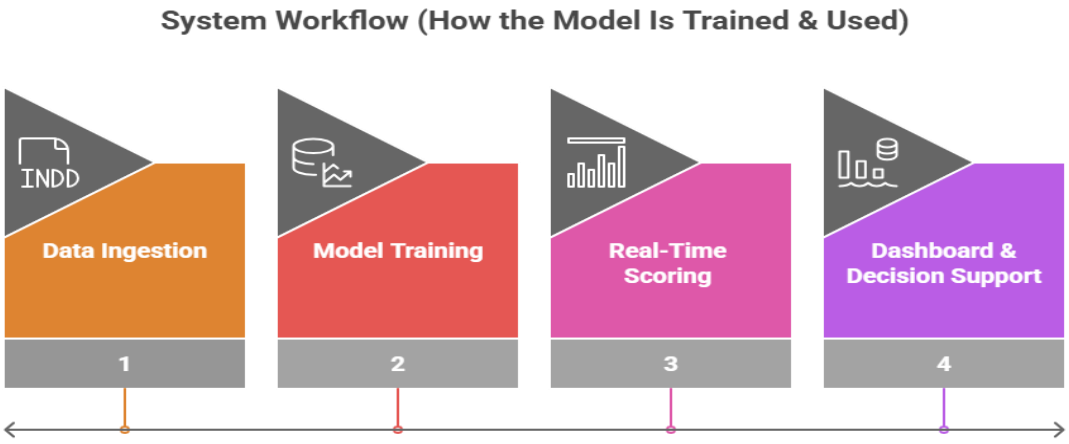
**Step 4: Anomaly Detection and Risk Flagging**
AAIOS applies anomaly detection models to identify districts exhibiting unusually high update rates, sudden enrolment spikes, or abnormal district–pin code mismatches. These anomalies are flagged separately for Bihar and Karnataka for further administrative review.

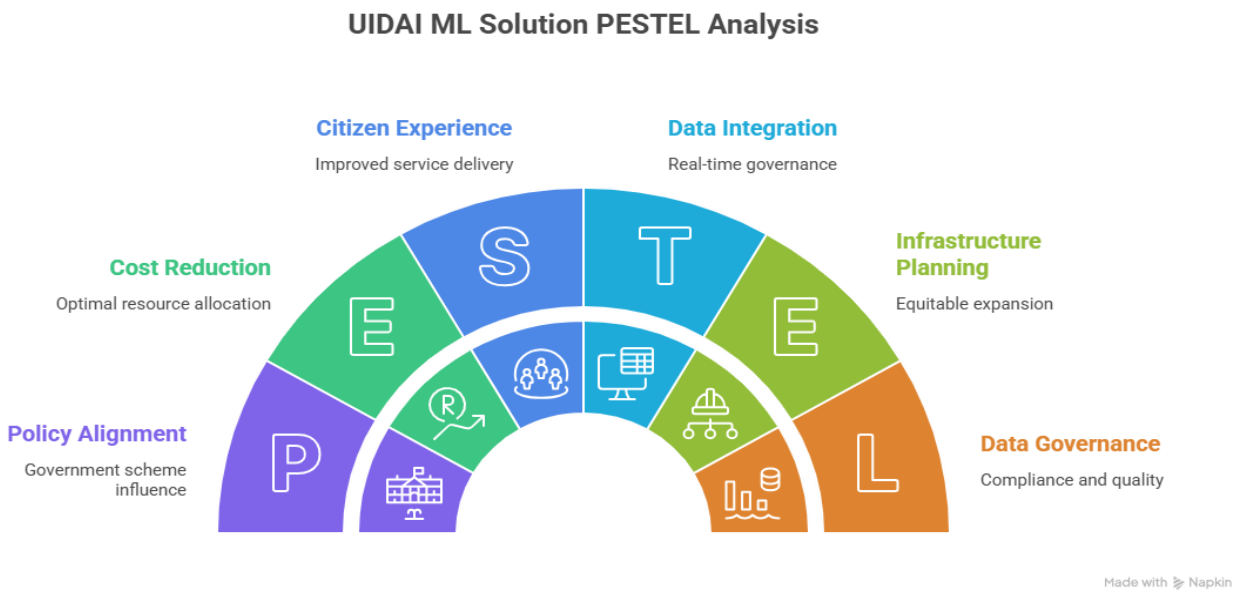
**Step 5: Predictive Resource Planning**
Based on historical and seasonal trends, the system predicts future enrolment and update demand at the district level. For Bihar, AAIOS recommends proactive scaling of enrolment centers and manpower during peak periods. For Karnataka, optimized resource reallocation is suggested to prevent underutilization.

**Step 6: Decision Support for UIDAI Authorities**
The final outputs are presented through dashboards and reports, enabling UIDAI officials to make informed, data-driven decisions on policy adjustments, infrastructure planning, and operational oversight while retaining full human control.

**Outcome**

This use-case demonstrates how AAIOS enables **state-specific intelligence**, supports **comparative regional analysis**, and transitions Aadhaar operations from reactive management to **predictive and optimized governance**.




## 7. Conclusion and Future Scope

This study demonstrates the significant role of large-scale data analytics in strengthening UIDAI's operational efficiency, policy planning, and governance capabilities. By systematically cleaning, standardizing, and analysing Aadhaar enrolment and demographic data, the study highlights how data-driven insights can

uncover regional disparities, temporal trends, and operational inefficiencies across states and districts.

The proposed analytical framework and AI-driven decision-support approach illustrate the potential of transforming Aadhaar's vast data ecosystem into actionable intelligence while maintaining strict adherence to privacy, security, and regulatory principles. The findings emphasize the importance of proactive planning over reactive management in large-scale public digital infrastructure.

**Future scope** of this work includes the development of advanced predictive models for enrolment demand forecasting, real-time dashboards for continuous monitoring, and robust anomaly detection systems for early identification of irregular patterns. Further enhancements may involve integration with external datasets such as census, migration, and socio-economic indicators to enable deeper contextual analysis and more informed policy decisions. With these extensions, the proposed system can evolve into a comprehensive, scalable, and intelligent governance support platform for UIDAI.